

ECOLE MILITAIRE POLYTECHNIQUE

Département d'Informatique

Rapport de TP

Module : Systèmes Distribués

Extension d'un système de messagerie distribuée

Nom : Bakouche
Prénom : Mahmoud
Groupe : RSD
Année universitaire : 2025–2026

20 avril 2026

Table des matières

Introduction générale	3
1 Intégration d'une base de données MySQL	4
1.1 Objectif de la partie	4
1.2 Conception de la base de données	4
1.3 Intégration avec l'application Java	5
1.4 Tests réalisés	5
1.5 Captures de la partie base de données	6
1.6 Analyse et discussion	15
2 Intégration d'une interface web de messagerie	16
2.1 Objectif de la partie	16
2.2 Architecture retenue	16
2.3 Fonctionnalités implémentées	17
2.4 Tests réalisés	17
2.5 Captures de la partie interface web	18
2.6 Analyse et discussion	24
3 Développement de clients de messagerie avec JavaMail	25
3.1 Objectif de la partie	25
3.2 Clients réalisés	25
3.3 Paramétrage utilisé	25
3.4 Fonctionnalités testées	26
3.5 Captures de la partie clients JavaMail	26
3.6 Analyse comparative POP3 / IMAP	31
3.7 Améliorations possibles	31

4 Scalabilité et Load Balancing	32
4.1 Objectif de la partie	32
4.2 Architecture déployée	32
4.3 Principe de fonctionnement	32
4.4 Scénarios de test réalisés	33
4.5 Captures de la partie load balancing	33
4.6 Analyse et discussion	40
 Conclusion générale	 42
 A Remarques pour Overleaf	 43

Introduction générale

Ce travail pratique s'inscrit dans le cadre du module de Systèmes Distribués et porte sur l'extension d'un système de messagerie distribuée déjà existant. L'objectif global consiste à reprendre une architecture de base puis à l'enrichir progressivement par l'intégration d'une base de données centralisée, d'une interface web, de clients de messagerie développés avec JavaMail, ainsi que d'un mécanisme de répartition de charge.

Le projet réalisé repose sur une implémentation Java comportant plusieurs composants complémentaires : un serveur d'authentification RMI, des serveurs SMTP, POP3 et IMAP, une base MySQL, une couche web fondée sur Spring Boot et un dispositif de load balancing à l'aide de NGINX et HAProxy. L'organisation du code montre également la présence de clients SMTP, POP3 et IMAP, d'interfaces graphiques d'administration et de scripts de lancement dédiés au déploiement de l'architecture.

Dans ce rapport, nous présentons essentiellement les parties correspondant aux sections 5, 6, 7 et 8 du sujet du TP, à savoir :

- l'intégration d'une base de données MySQL ;
- l'intégration d'une interface web de messagerie ;
- le développement de clients de messagerie avec JavaMail ;
- la mise en place d'une architecture scalable avec load balancing.

1

Intégration d'une base de données MySQL

1.1 Objectif de la partie

Cette partie a pour objectif de remplacer le stockage local par une base de données MySQL centralisée afin d'améliorer la cohérence, la persistance et la scalabilité du système de messagerie. Le but est de centraliser la gestion des comptes utilisateurs ainsi que le stockage des emails, tout en permettant aux différents serveurs du système de partager la même source de données.

1.2 Conception de la base de données

La base de données mise en place contient deux tables principales :

- **users** : cette table stocke les informations relatives aux utilisateurs, notamment un identifiant unique, un nom d'utilisateur, un mot de passe haché, une date de création et un statut ;
- **emails** : cette table stocke les messages envoyés ou reçus avec les métadonnées associées telles que l'expéditeur, le destinataire, le sujet, le contenu, l'état de lecture et la date d'envoi.

Le fichier `init.sql` initialise la base `messagerie_db`, crée les tables nécessaires et définit les procédures stockées `store_email`, `fetch_emails`, `delete_email` et `update_password`. Cette organisation permet de simplifier l'accès aux données depuis les différents services Java tout en gardant une logique de persistance centralisée.

1.3 Intégration avec l'application Java

L'accès à MySQL est assuré par la classe `DatabaseManager`. Cette classe joue le rôle de couche d'accès aux données et regroupe les opérations principales : authentification des utilisateurs, création de compte, mise à jour, suppression, stockage des emails, récupération des messages et mise à jour de l'état de lecture.

L'application utilise JDBC pour communiquer avec MySQL. L'approche adoptée repose sur :

- des **PreparedStatement** pour les requêtes paramétrées ;
- des **CallableStatement** pour l'appel des procédures stockées ;
- une classe singleton pour centraliser les connexions à la base.

Cette intégration a permis de faire de MySQL le noyau de persistance partagé entre le serveur d'authentification, les services de messagerie et l'interface web.

1.4 Tests réalisés

Les tests menés dans cette partie ont porté sur les points suivants :

1. création et vérification de la base de données ;
2. insertion et consultation des utilisateurs ;
3. insertion d'emails via le service SMTP ;
4. consultation et suppression des emails depuis POP3 et IMAP ;
5. vérification de l'état `is_read` pour la lecture des messages.

Les captures suivantes illustrent la conception de la base, les tables, les données insérées et les différents tests réalisés.

1.5 Captures de la partie base de données

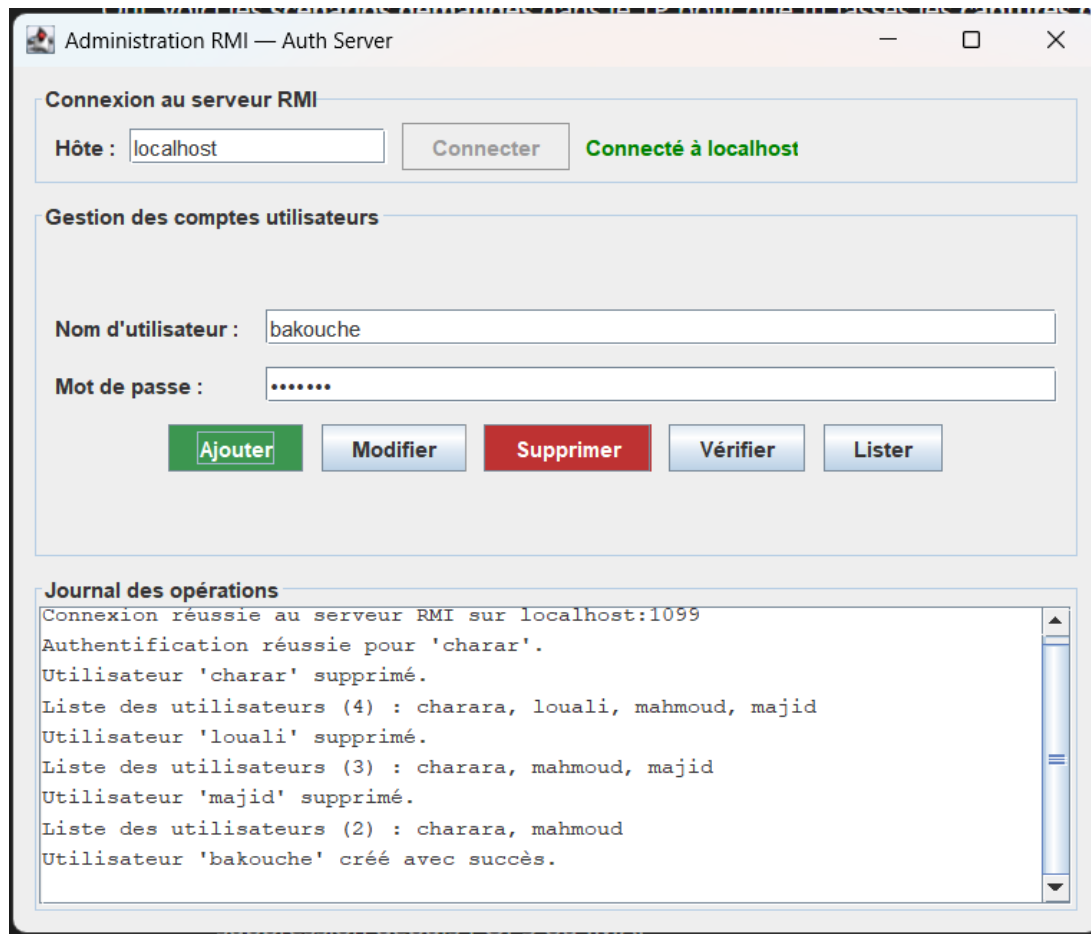


FIGURE 1.1 – Création de la base de données et exécution du script d'initialisation.

	id	username	password_hash	created_at	status
▶	1	mahmoud	77459b9b941bcb4714d0c121313c900ecf30541...	2026-04-09 19:44:46	ACTIVE
	2	charara	483029d526219f816e8e8f6a9de07b422633dba...	2026-04-10 02:24:00	ACTIVE
	6	bakouche	075095892732ff49cdc485570408ebce0bf3abb...	2026-04-20 08:07:27	ACTIVE
✱	NULL	NULL	NULL	NULL	NULL

FIGURE 1.2 – Structure générale de la base de données.

```

DROP PROCEDURE IF EXISTS authenticate_user //
CREATE PROCEDURE authenticate_user(IN p_username VARCHAR(100), IN p_password VARCHAR(255), OUT p_is_valid BOOL
BEGIN
    SELECT COUNT(*) INTO @cnt FROM users WHERE username = p_username AND password_hash = p_password;
    IF @cnt > 0 THEN
        SET p_is_valid = TRUE;
    ELSE
        SET p_is_valid = FALSE;
    END IF;
END //

```

FIGURE 1.3 – Table des utilisateurs après insertion des premiers comptes.

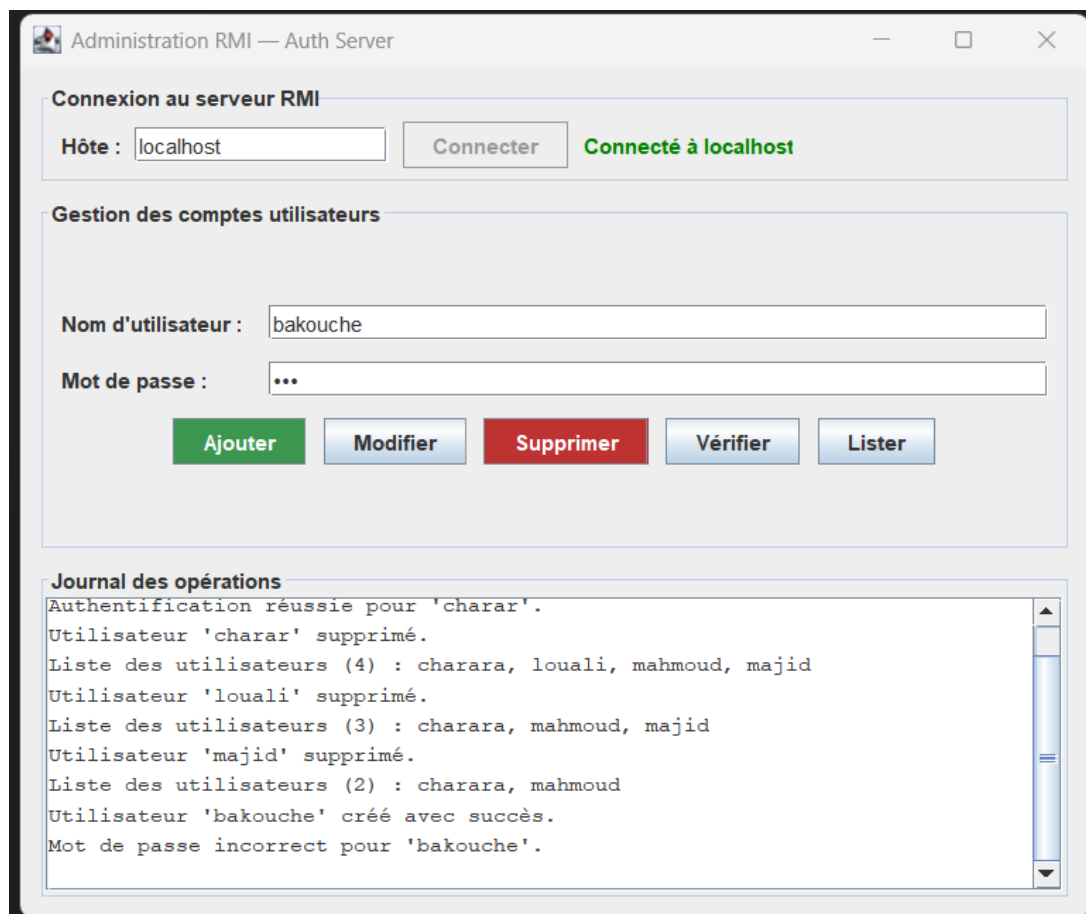


FIGURE 1.4 – Table des emails et vérification des messages stockés.

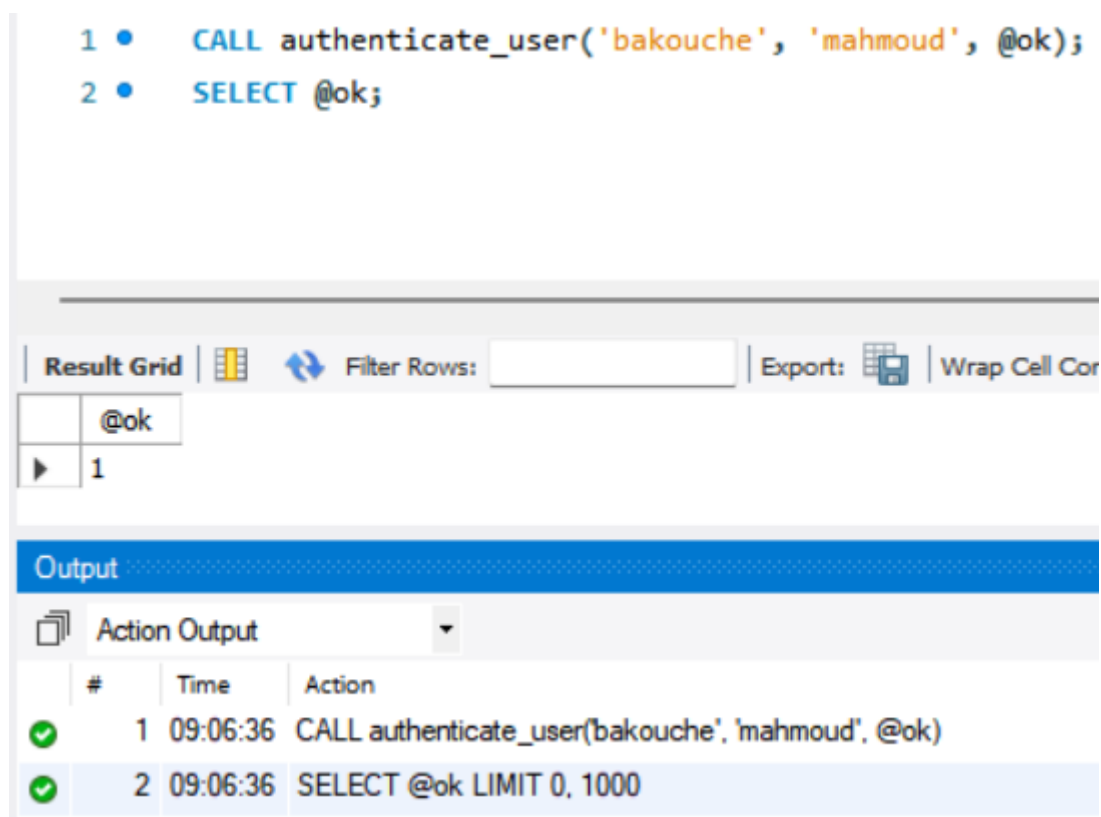


FIGURE 1.5 – Test d’insertion d’un email dans la base.

The screenshot displays a database client interface. At the top, two SQL statements are entered in a query editor:

```
1 • CALL authenticate_user('bakouche', '123', @ok);
2 • SELECT @ok;
```

Below the editor, the 'Result Grid' tab is active, showing a single column named '@ok' with a value of 0. The grid has a header row and one data row.

Below the result grid, the 'Output' tab is active, showing a table of actions. The table has four columns: '#', 'Time', and 'Action'. There are four rows of actions, each marked with a green checkmark in the first column.

#	Time	Action
✓ 1	09:06:36	CALL authenticate_user('bakouche', 'mahmoud', @ok)
✓ 2	09:06:36	SELECT @ok LIMIT 0, 1000
✓ 3	09:07:34	CALL authenticate_user('bakouche', '123', @ok)
✓ 4	09:07:34	SELECT @ok LIMIT 0, 1000

FIGURE 1.6 – Test de récupération des emails pour un utilisateur.

```

220 smtp.example.com Service Ready
EHLO localhost
250 Hello localhost
MAIL FROM:<mahmoud@gmail.com>
250 OK
RCPT TO:<charara@gmail.com>
250 OK
DATA
354 Start mail input; end with <CRLF>.<CRLF>
123456789
.
250 OK: Message accepted for delivery
QUIT
221 smtp.example.com Service closing transmission channel

Connection to host lost.

```

FIGURE 1.7 – Test de suppression d’un message depuis la base.

```
1 select * from emails
```

Result Grid							
		Filter Rows:		Edit:		Export/Import:	
id	sender	recipient	subject	content	is_read	created_at	
19	mahmoud	charara	Email	123456789	0	2026-04-20 11:24:20	
NULL	NULL	NULL	NULL	NULL	NULL	NULL	

FIGURE 1.8 – Validation des procédures stockées.

```
PASS 2004
+OK Password accepted
LIST
+OK 1 messages
1 86
.
RETR 1
+OK 86 octets
From: mahmoud
.
RETR 1
+OK 86 octets
From: mahmoud
To: charara
Subject: Email
Date: 2026-04-20 11:24:20.0

123456789
.
QUIT
+OK POP3 server signing off
```

FIGURE 1.9 – Vérification de l'état de lecture des messages.

```
* OK IMAP4rev2 Service Ready
a1 LOGIN charara 2004
a1 OK LOGIN completed
a2 SELECT INBOX
* 1 EXISTS
* FLAGS (\Seen)
a2 OK [READ-WRITE] SELECT completed
a3 FETCH 1 BODY[]
* 1 FETCH (BODY[] {86}
From: mahmoud
To: charara
Subject: Email
Date: 2026-04-20 11:24:20.0

123456789
)
a3 OK FETCH completed
a4 LOGOUT
* BYE IMAP4rev2 Server logging out
a4 OK LOGOUT completed

Connection to host lost.
```

FIGURE 1.10 – Test de l'authentification utilisateur à partir de MySQL.

```
+OK POP3 server ready
USER CHARARA
+OK User accepted
PASS 2004
+OK Password accepted
LIST
+OK 1 messages
1 86
.
DELE 1
+OK Message marked for deletion
QUIT
+OK POP3 server signing off

Connection to host lost.
█
```

FIGURE 1.11 – Validation de la cohérence entre les services et la base.

```
1 select * from emails
```

Result Grid | Filter Rows: | Edit:   

id	sender	recipient	subject	content	is_read	created_at
NULL	NULL	NULL	NULL	NULL	NULL	NULL

FIGURE 1.12 – Exemple de données persistantes partagées entre les serveurs.

```
+OK POP3 server ready
USER ' OR '1'='1
-ERR User not found
```

FIGURE 1.13 – Test de mise à jour ou de correction des informations utilisateur.

```
CALL authenticate_user('bakouche', 'mahmoud', @ok);
SELECT @ok;
```

Automatic context help is disabled. Use the toolbar to manually get help. [Context Help](#) [Snippets](#)

Time	Action	Message	Duration / Fetch
12:28:09	CALL authenticate_user('bakouche', 'mahmoud', @ok)	1 row(s) affected	0.000 sec
12:28:09	SELECT @ok LIMIT 0, 1000	1 row(s) returned	0.000 sec / 0.000 sec

FIGURE 1.14 – Validation finale du schéma et des données.

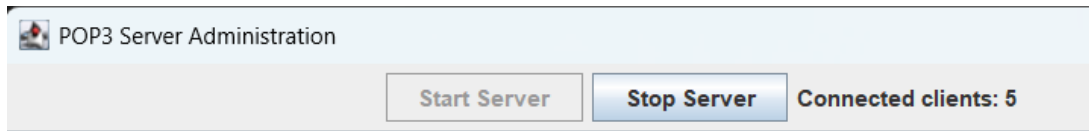


FIGURE 1.15 – Résultat final de la partie base de données.

1.6 Analyse et discussion

L'intégration de MySQL a permis de renforcer la cohérence globale du système. Au lieu d'utiliser un stockage local par fichiers, tous les services accèdent désormais à une base commune, ce qui facilite la synchronisation des données dans une architecture distribuée.

Parmi les points positifs observés, on peut citer :

- la persistance centralisée des utilisateurs et des emails ;
- la simplification des opérations d'administration ;
- la possibilité de connecter plusieurs services à la même base ;
- une meilleure préparation à la montée en charge.

Cependant, certaines limites subsistent :

- les paramètres de connexion à la base sont encore codés en dur dans certaines classes ;
- le mot de passe MySQL doit être externalisé dans un fichier de configuration sécurisé ;
- une gestion plus avancée des exceptions SQL serait souhaitable ;
- des tests de performance plus poussés pourraient être ajoutés.

2

Intégration d'une interface web de messagerie

2.1 Objectif de la partie

L'objectif de cette partie est d'offrir un accès web au système de messagerie à travers un navigateur. L'utilisateur doit pouvoir s'authentifier, consulter sa boîte de réception, lire ses messages, supprimer des emails et envoyer de nouveaux messages sans interagir directement avec les protocoles bas niveau.

2.2 Architecture retenue

La solution web repose sur une application Spring Boot. Le composant principal est la classe `MailController`, qui expose des points d'accès REST sous le préfixe `/api`. L'architecture suit une séparation claire entre :

- la couche présentation : fichiers statiques `index.html`, `app.js` et `style.css` ;
- la couche service web : application Spring Boot ;
- la couche d'authentification : serveur RMI ;
- la couche de persistance : base MySQL ;
- la couche de messagerie : SMTP pour l'envoi et accès aux messages via la base.

L'application web ne dialogue pas directement avec la base pour l'authentification. Lors du login, elle contacte le serveur d'authentification via RMI puis génère un jeton JWT pour protéger les appels suivants. Les requêtes vers les messages vérifient ensuite l'en-tête `Authorization` et extraient le nom d'utilisateur depuis le token.

2.3 Fonctionnalités implémentées

Les principales fonctionnalités réalisées sont les suivantes :

- connexion utilisateur avec login/mot de passe ;
- création de compte ;
- consultation des messages reçus ;
- consultation des messages envoyés ;
- marquage d'un email comme lu ;
- suppression d'un message ;
- envoi d'un email à travers le service SMTP distant.

L'envoi d'un message via l'interface web se fait en ouvrant une socket vers le service SMTP. L'application joue alors le rôle d'un client SMTP et envoie successivement les commandes HELO, MAIL FROM, RCPT TO, DATA puis QUIT. Cette approche respecte la logique du sujet qui impose de faire transiter les opérations par les services de messagerie existants.

2.4 Tests réalisés

Les tests fonctionnels ont couvert plusieurs scénarios :

1. authentification valide ;
2. authentification invalide ;
3. accès à la boîte de réception ;
4. lecture d'un message ;
5. envoi d'un email via le formulaire web ;
6. suppression d'un message ;
7. tentative d'accès non autorisé sans token.

2.5 Captures de la partie interface web

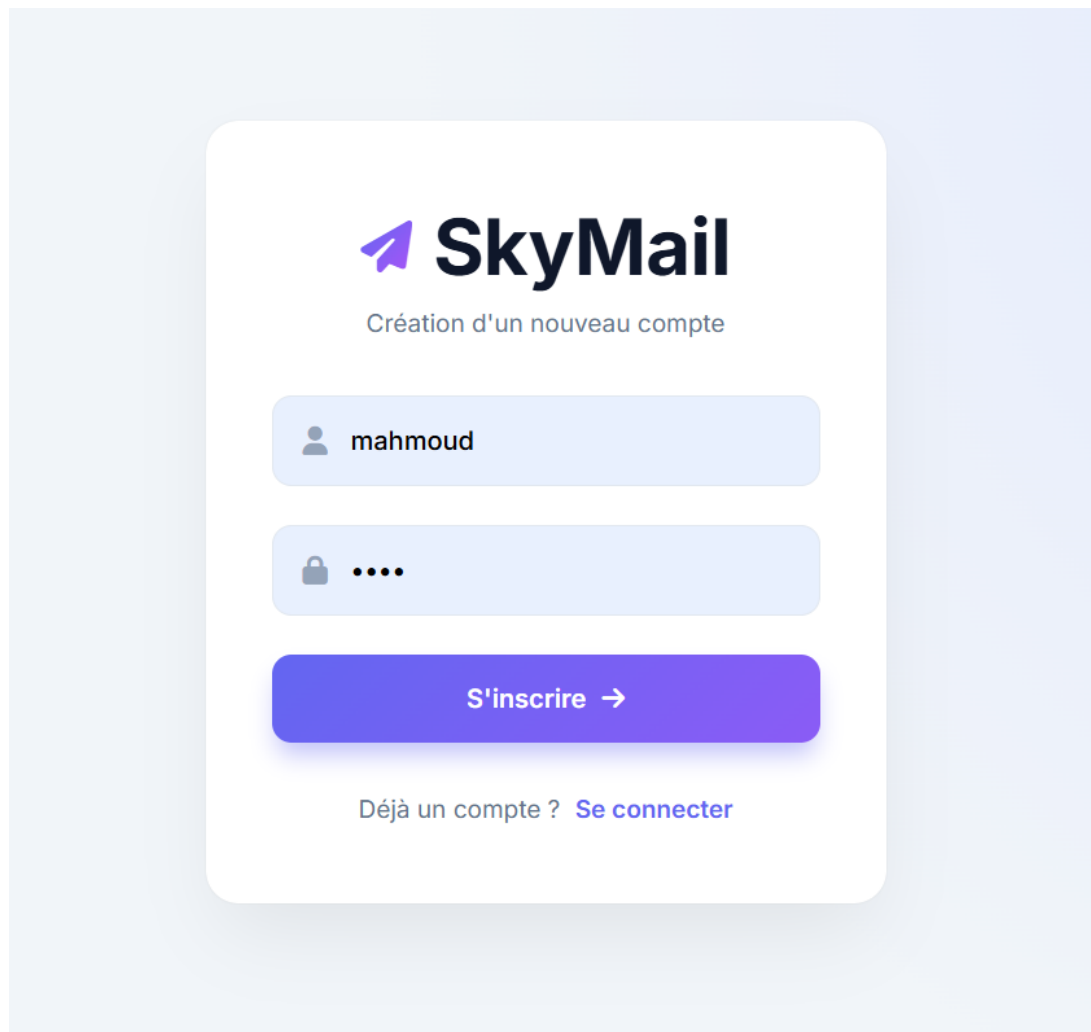


FIGURE 2.1 – Page d’authentification de l’interface web.



FIGURE 2.2 – Connexion valide et accès à la boîte de réception.

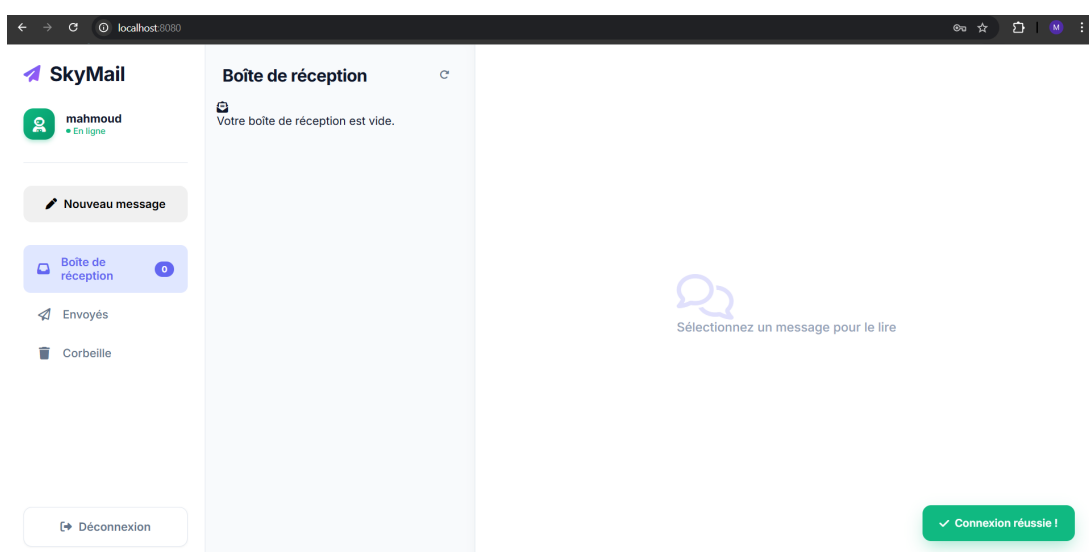


FIGURE 2.3 – Connexion invalide et message d'erreur.

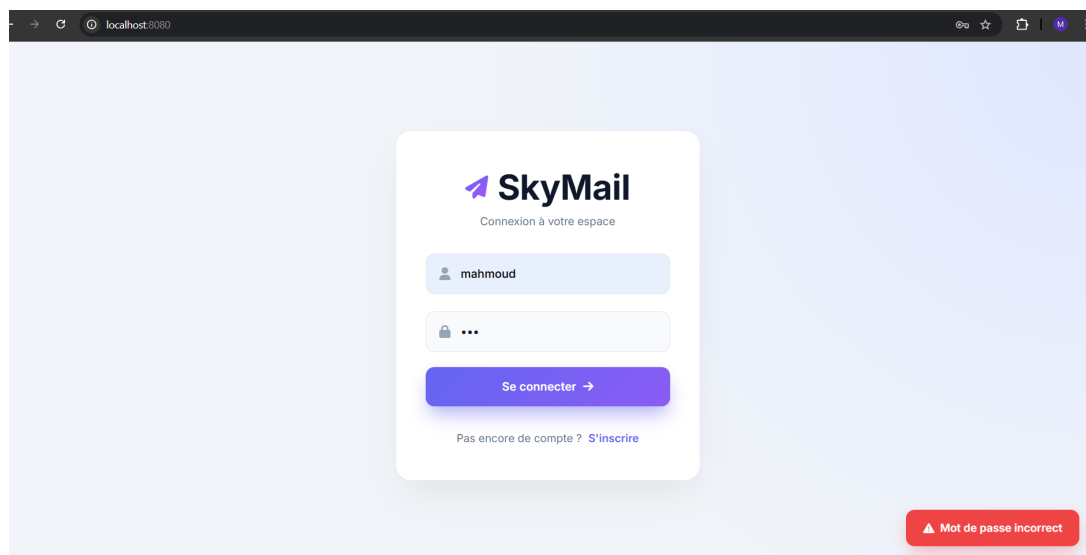


FIGURE 2.4 – Affichage de la liste des emails reçus.

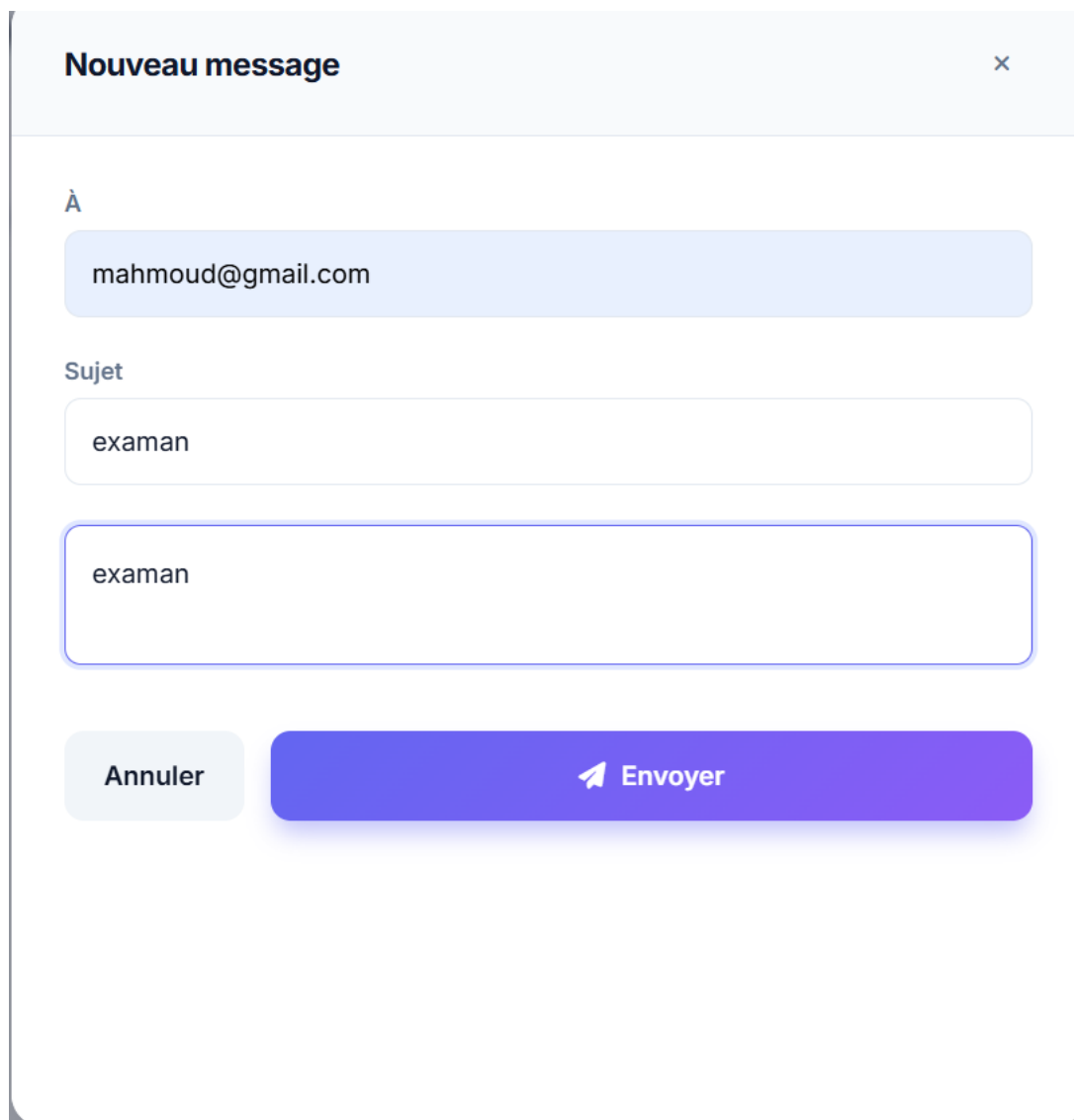


FIGURE 2.5 – Lecture d'un message depuis l'interface web.

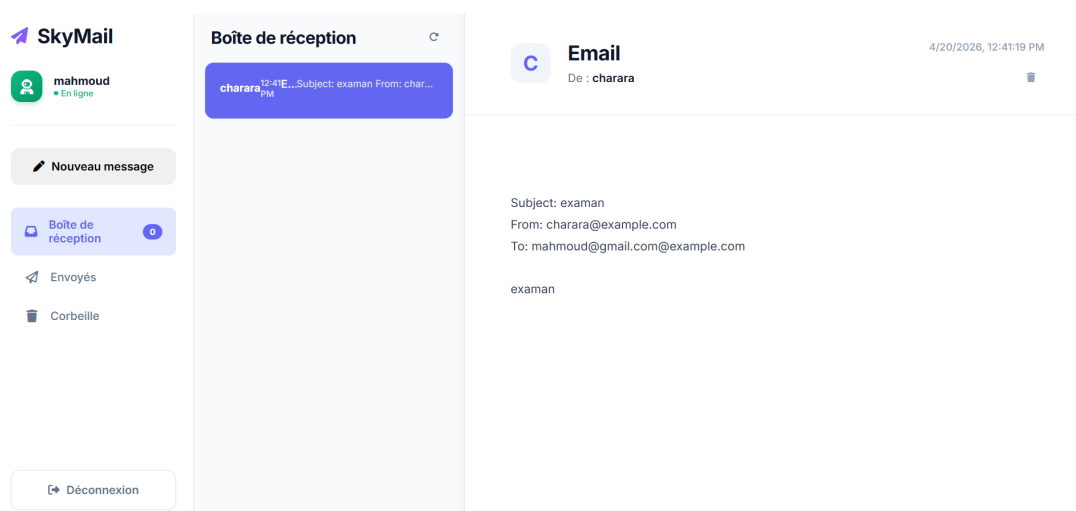


FIGURE 2.6 – Marquage d'un message comme lu.

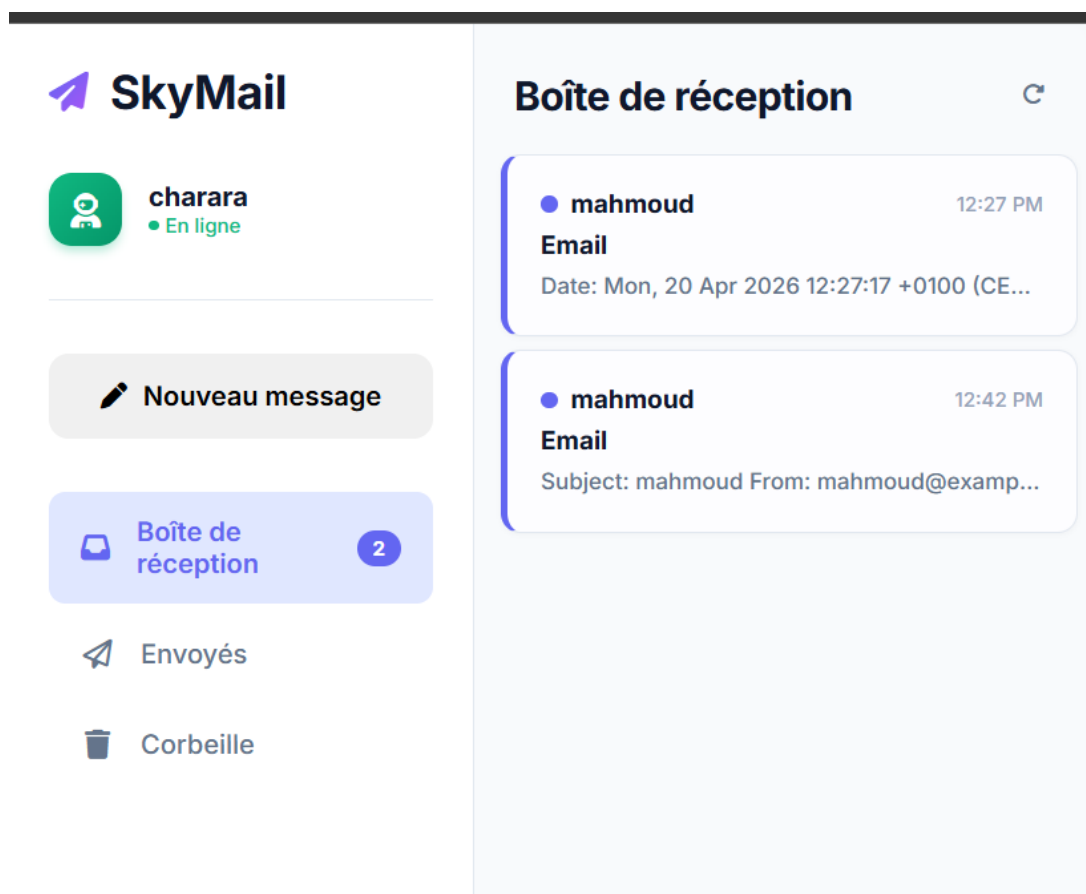


FIGURE 2.7 – Formulaire d'envoi d'un nouvel email.

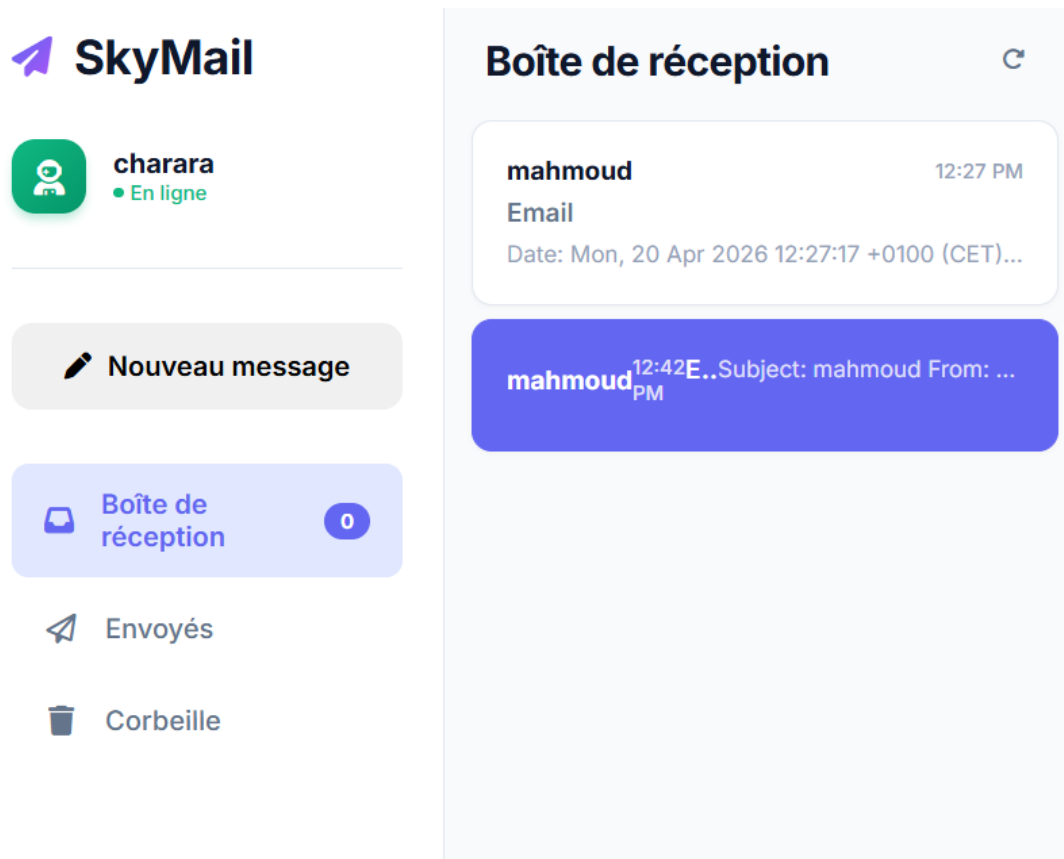


FIGURE 2.8 – Validation d'un envoi d'email via l'interface web.

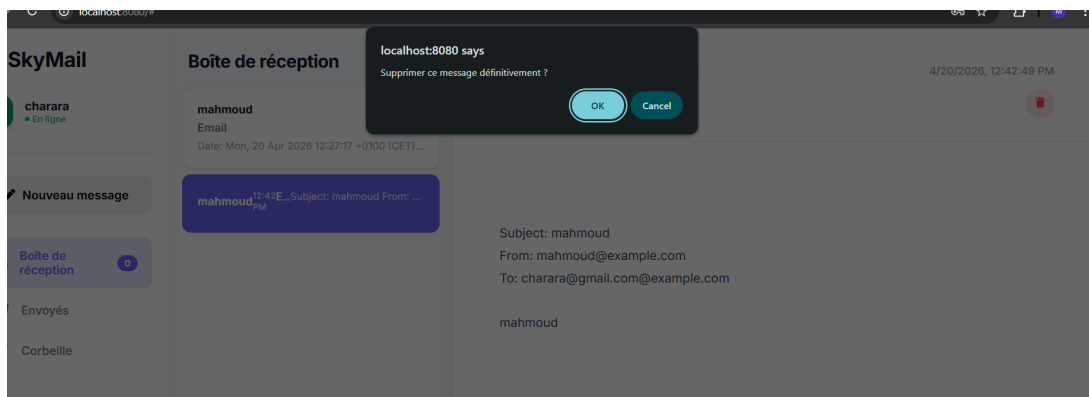


FIGURE 2.9 – Suppression d'un message depuis l'interface.

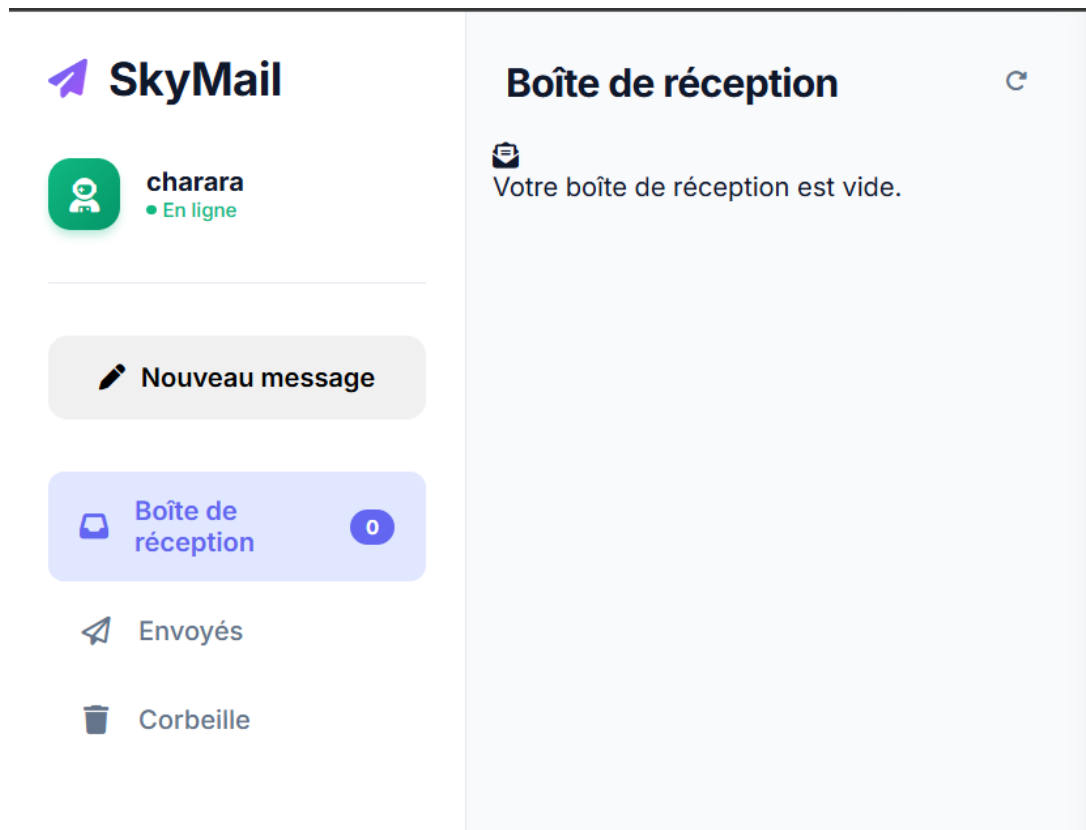


FIGURE 2.10 – Consultation des messages envoyés ou test complémentaire de l’API.

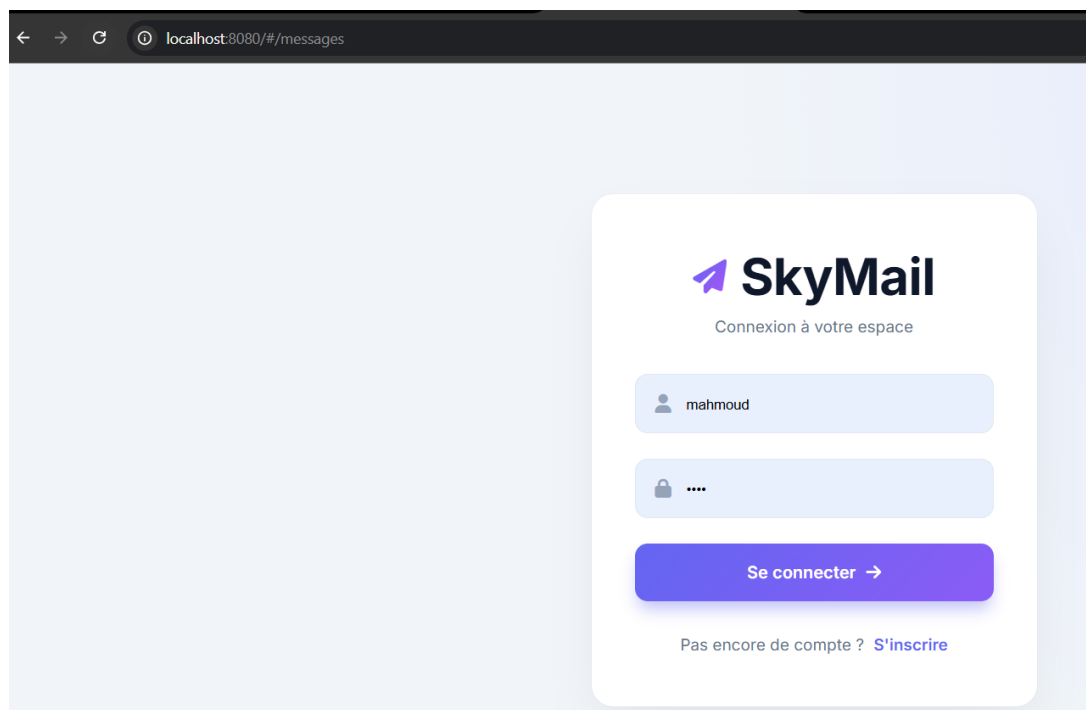


FIGURE 2.11 – Validation finale du bon fonctionnement global de l’interface web.

2.6 Analyse et discussion

L'interface web apporte une couche de présentation moderne et facilite l'utilisation du système. Elle illustre correctement la séparation entre front-end, API applicative, authentification et services distribués.

Les avantages principaux sont :

- une meilleure accessibilité du système ;
- une utilisation simple depuis un navigateur ;
- une protection des routes à l'aide d'un token JWT ;
- une intégration cohérente avec le serveur d'authentification et le service SMTP.

Parmi les améliorations possibles, on peut proposer :

- l'ajout d'une pagination pour les listes de messages ;
- la recherche multi-critères ;
- la gestion des pièces jointes ;
- une meilleure validation des adresses destinataires ;
- l'utilisation du protocole HTTPS et d'une sécurisation plus avancée des jetons.

3

Développement de clients de messagerie avec JavaMail

3.1 Objectif de la partie

L'objectif de cette partie est de développer des clients capables d'interagir avec les serveurs SMTP, POP3 et IMAP du projet sans réimplémenter les protocoles bas niveau. Pour cela, la bibliothèque JavaMail (Jakarta Mail) a été utilisée.

3.2 Clients réalisés

Le projet contient plusieurs clients :

- **SmtplibClient** pour l'envoi de messages ;
- **Pop3Client** pour la consultation simple et la suppression ;
- **ImapClient** pour la lecture avancée et la recherche ;
- **MailClientGUI** qui unifie les trois usages dans une interface graphique unique.

L'interface graphique unifiée constitue l'élément le plus intéressant de cette partie. Elle propose des onglets distincts pour l'envoi SMTP, la consultation POP3 et la consultation IMAP. L'utilisateur peut ainsi tester plusieurs modes d'accès à la messagerie depuis une même application.

3.3 Paramétrage utilisé

Les clients utilisent les ports suivants :

- SMTP : port 25 ;
- POP3 : port 110 ;

— IMAP : port 1430.

Ces choix correspondent à l'architecture mise en place avec le load balancer, qui redirige ensuite vers les différentes instances backend lorsque le déploiement complet est actif.

3.4 Fonctionnalités testées

Les tests ont porté sur :

1. l'authentification avec identifiants valides ;
2. l'échec d'authentification avec mot de passe incorrect ;
3. l'envoi d'un email via SMTP ;
4. la lecture et la suppression via POP3 ;
5. la lecture, le marquage et la recherche via IMAP ;
6. la gestion des erreurs lorsque le serveur est indisponible.

3.5 Captures de la partie clients JavaMail

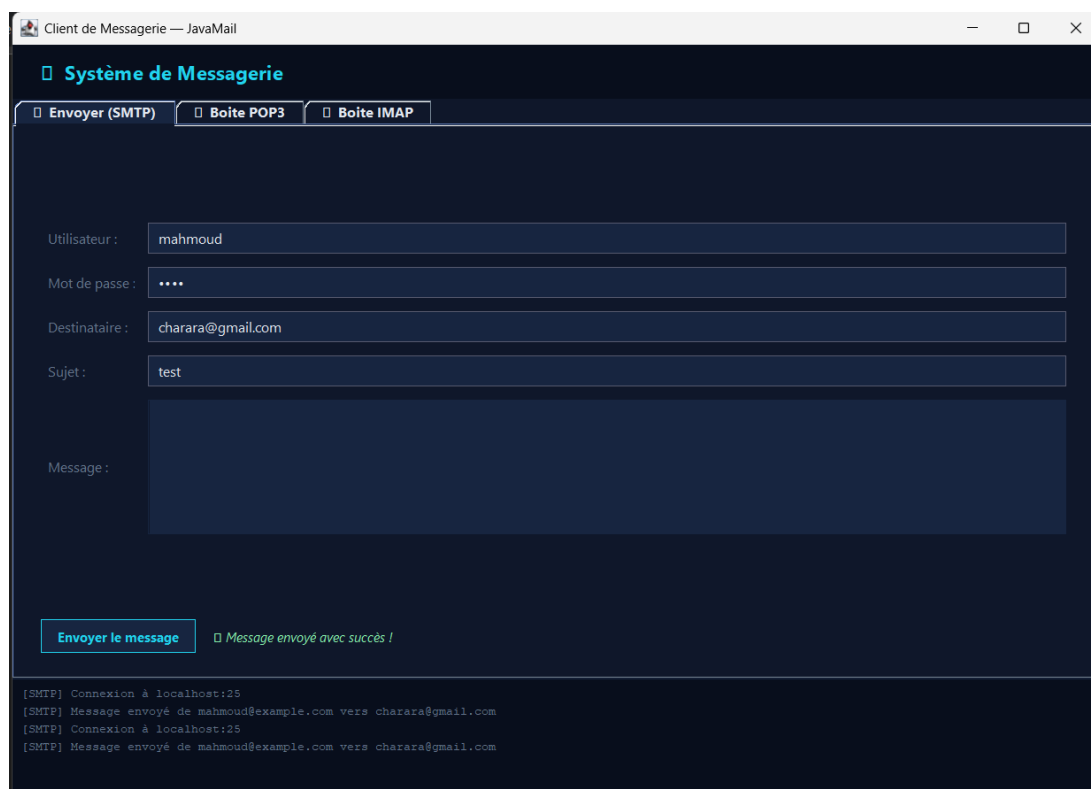


FIGURE 3.1 – Lancement du client JavaMail ou de l'interface graphique unifiée.

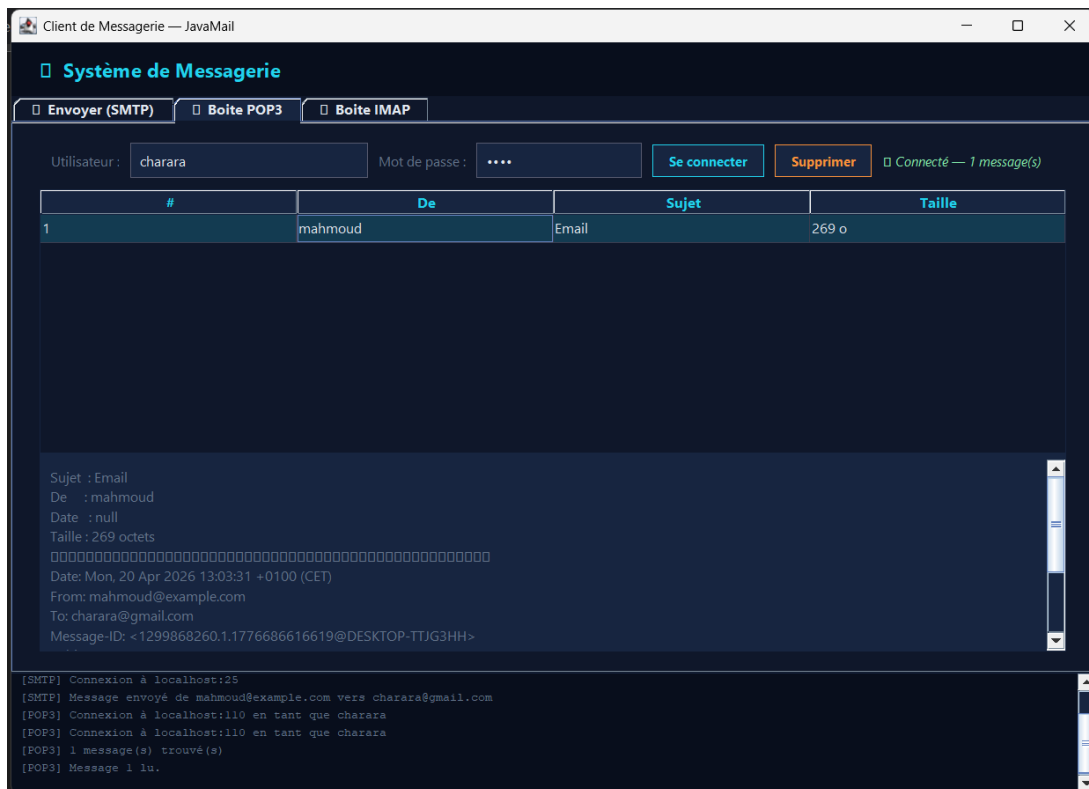


FIGURE 3.2 – Authentification de l'utilisateur dans le client SMTP.

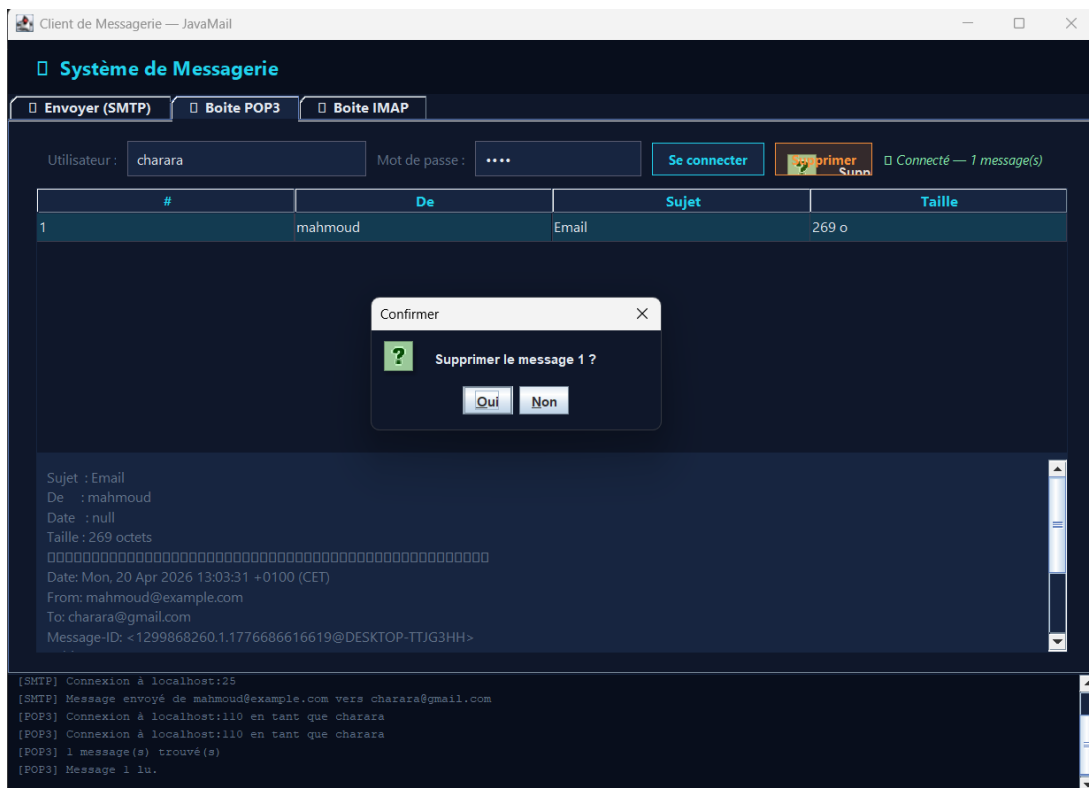


FIGURE 3.3 – Envoi d'un email via le client SMTP.

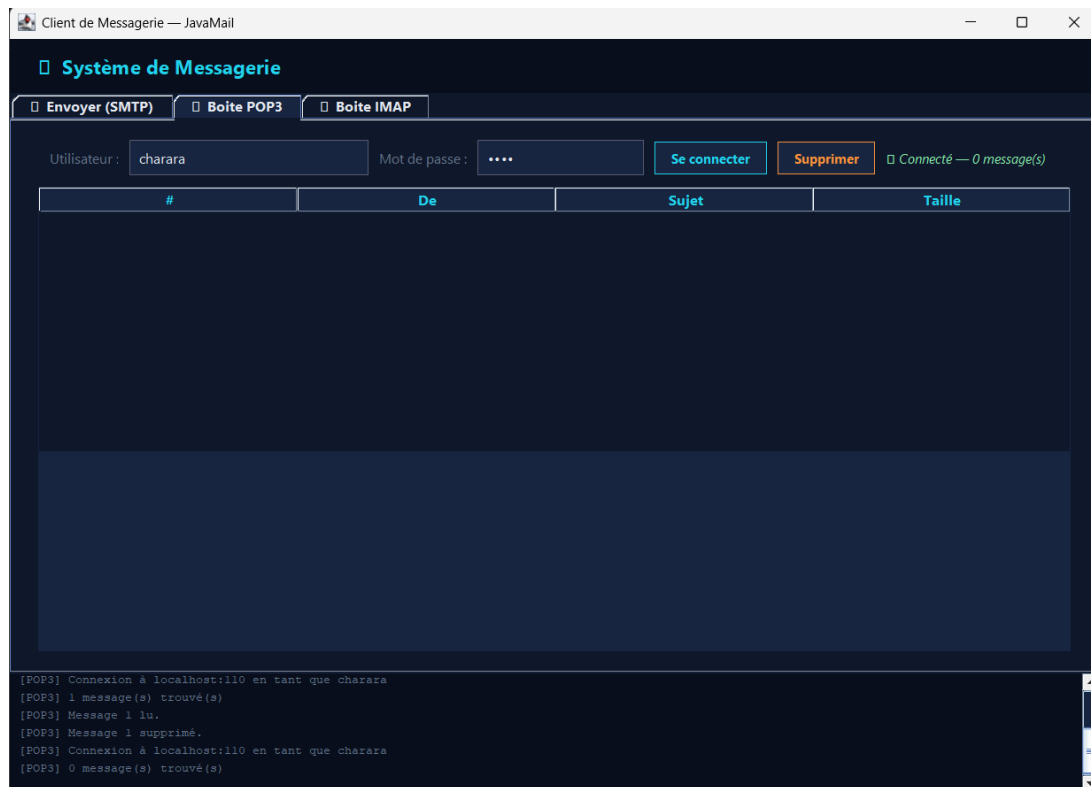


FIGURE 3.4 – Affichage des messages via POP3.

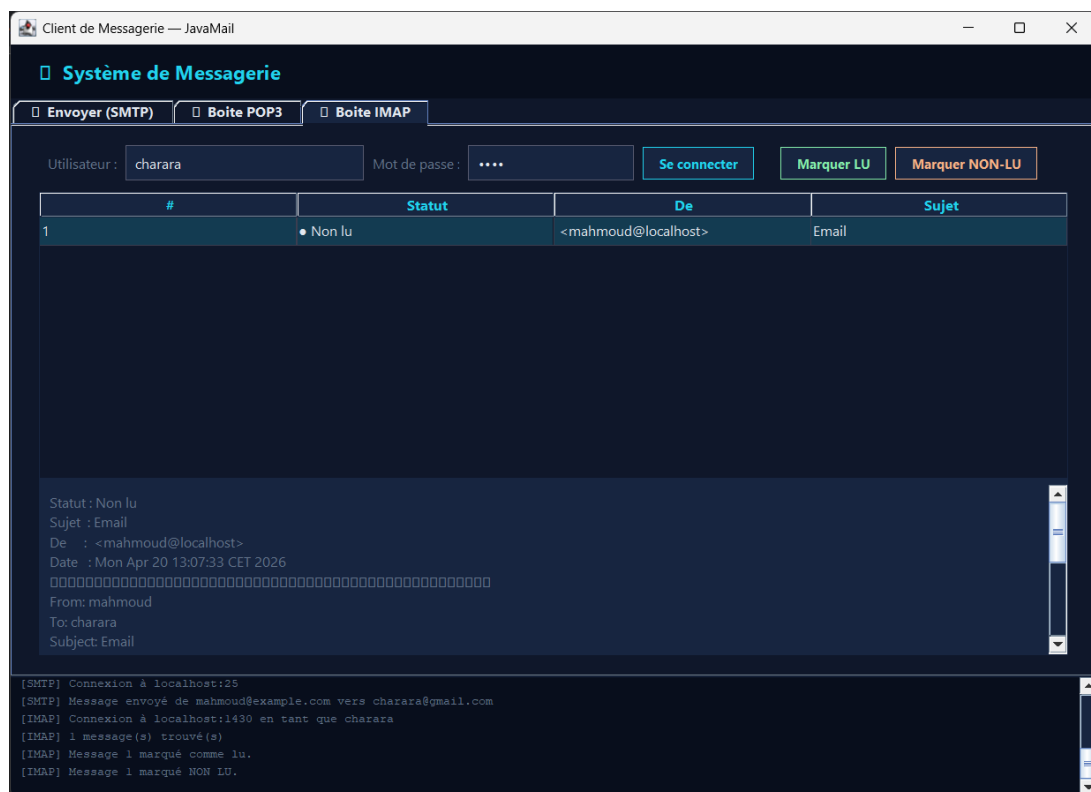


FIGURE 3.5 – Lecture d'un message dans le client POP3.

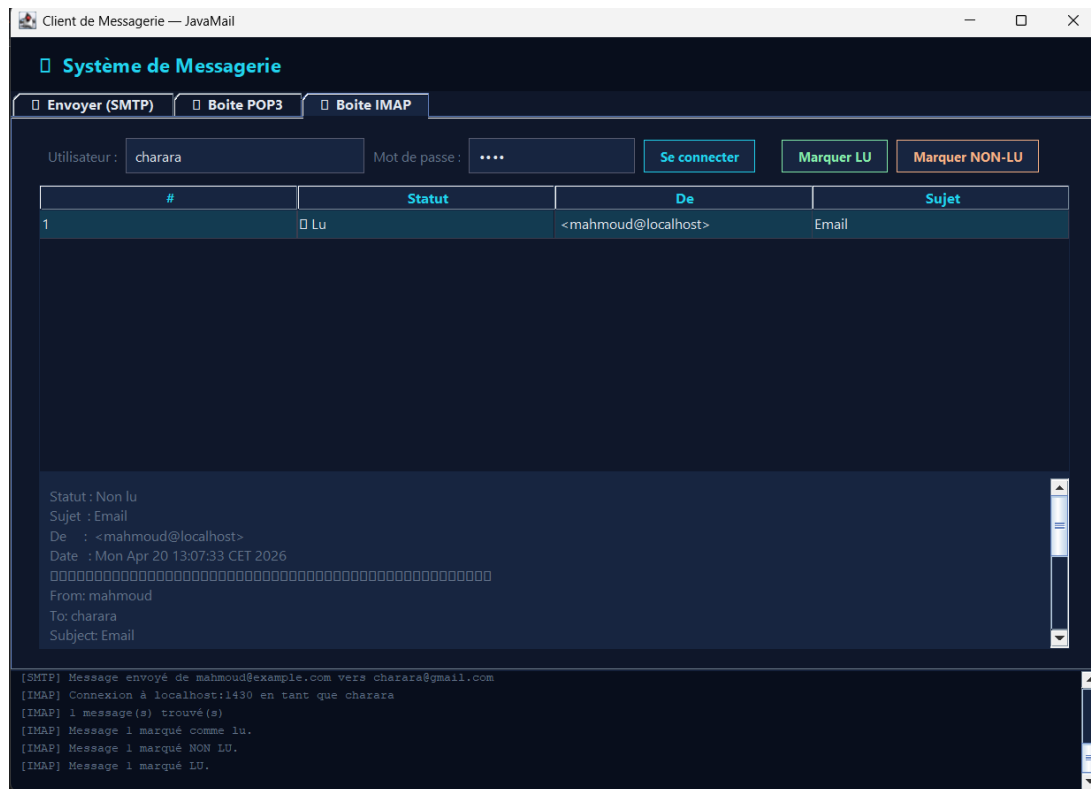


FIGURE 3.6 – Suppression d'un message via POP3.

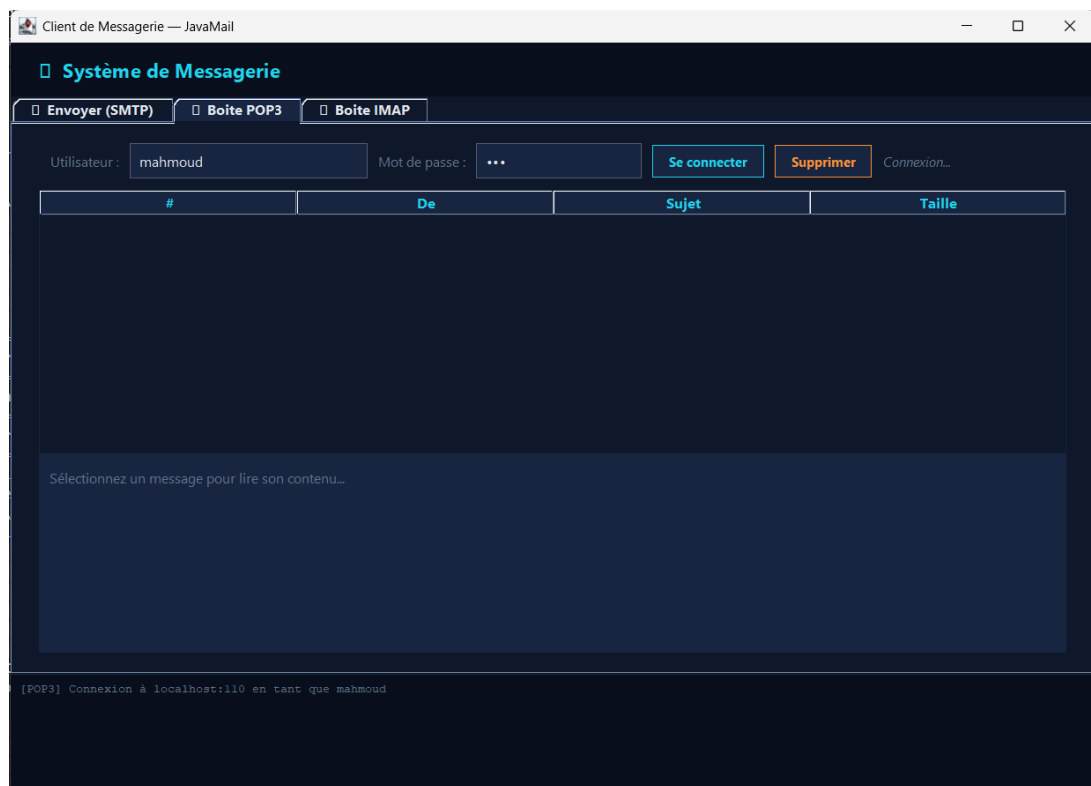


FIGURE 3.7 – Consultation de la boîte IMAP.

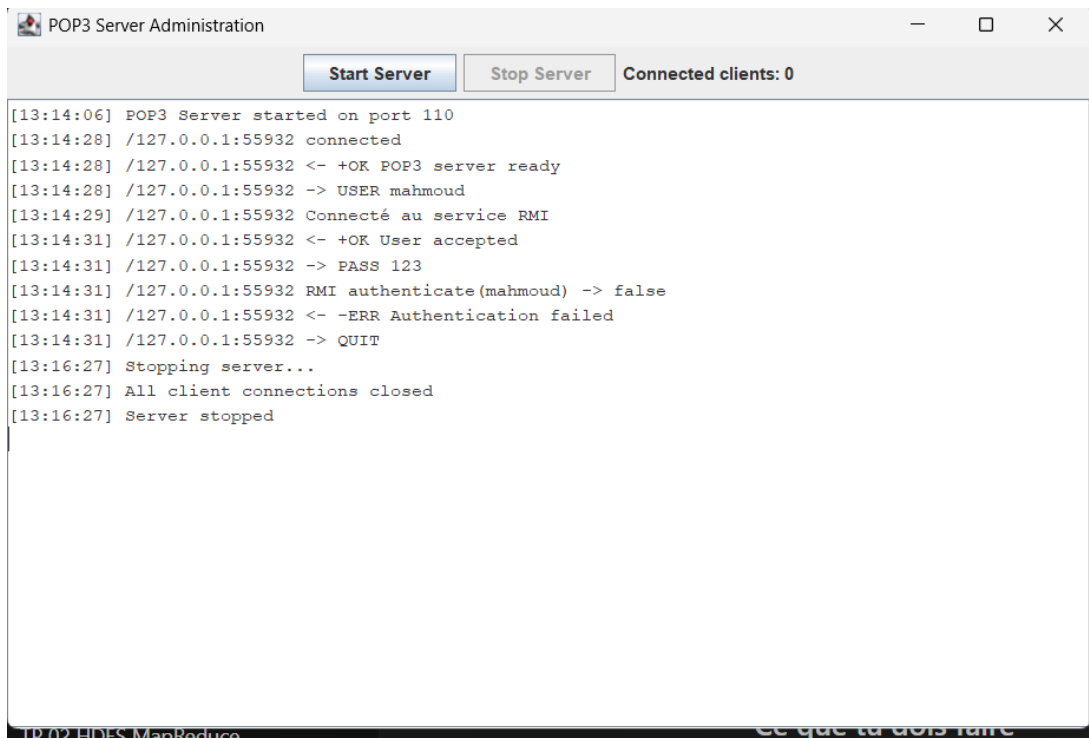


FIGURE 3.8 – Recherche ou gestion avancée des messages avec IMAP.

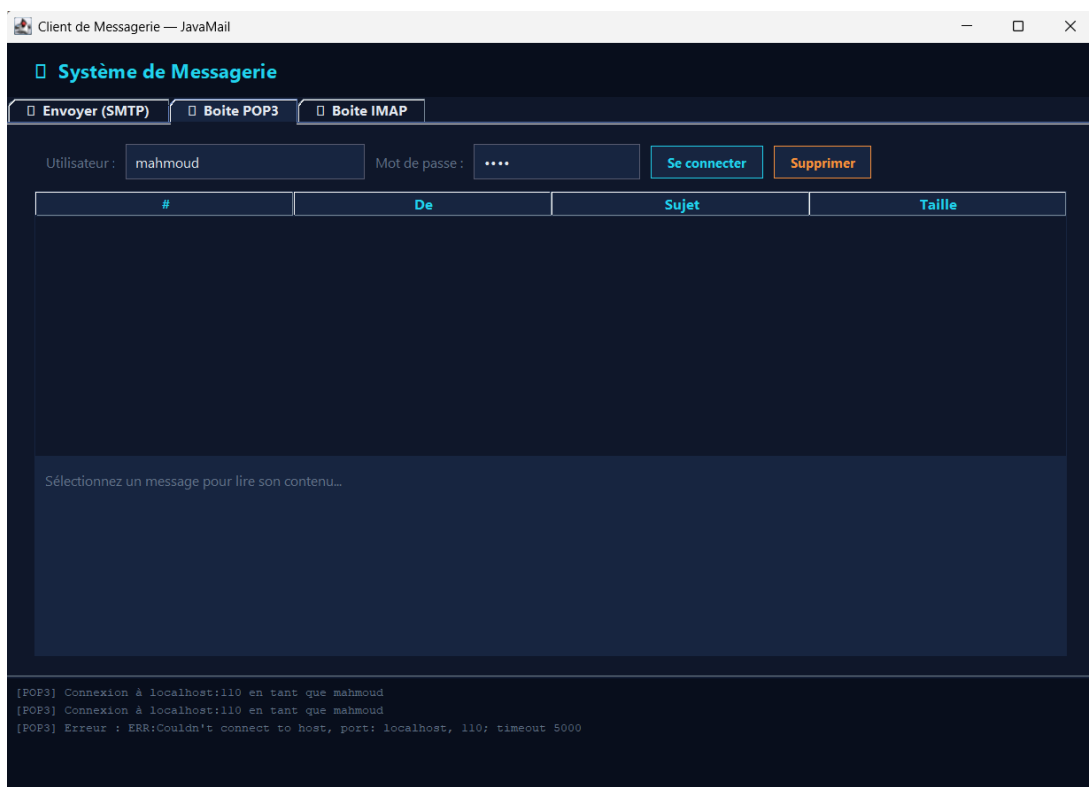


FIGURE 3.9 – Validation finale des clients JavaMail et de leur intégration.

3.6 Analyse comparative POP3 / IMAP

L'expérimentation a permis de mettre en évidence plusieurs différences importantes entre POP3 et IMAP.

POP3 offre une logique simple de consultation et de téléchargement des messages. Il est adapté à des usages basiques mais reste limité en matière de synchronisation et d'opérations avancées.

IMAP, en revanche, permet une interaction plus riche avec la boîte aux lettres : lecture distante, recherche, gestion de l'état des messages et synchronisation plus fine. Cela le rend mieux adapté à une utilisation moderne de la messagerie.

L'utilisation de JavaMail a été très utile car elle a permis de se concentrer sur l'intégration et les tests fonctionnels plutôt que sur la réimplémentation technique des protocoles.

3.7 Améliorations possibles

Parmi les améliorations possibles pour cette partie, on peut citer :

- l'ajout du support des pièces jointes ;
- l'affichage HTML des emails ;
- l'ajout du chiffrement TLS ;
- une ergonomie encore meilleure de l'interface graphique ;
- la gestion de dossiers multiples côté IMAP.

4

Scalabilité et Load Balancing

4.1 Objectif de la partie

Cette partie vise à améliorer la disponibilité et la capacité de montée en charge du système en dupliquant les services de messagerie et en plaçant un répartiteur de charge en frontal. Le principe adopté repose sur une stratégie *Round Robin*.

4.2 Architecture déployée

Le projet contient une configuration complète de load balancing avec :

- deux instances SMTP sur les ports 2526 et 2527 ;
- deux instances POP3 sur les ports 1111 et 1112 ;
- deux instances IMAP sur les ports 1431 et 1432 ;
- deux instances Web sur les ports 8080 et 8081 ;
- un frontal NGINX ou HAProxy pour répartir les requêtes ;
- une base de données commune partagée par l'ensemble des nœuds.

Le script `run_lb.bat` automatise une grande partie du lancement des différents composants. Le fichier `nginx.conf` montre que la répartition est effectuée en Round Robin aussi bien pour les flux TCP (SMTP, POP3, IMAP) que pour la couche HTTP. Le fichier `haproxy.cfg` fournit une alternative équivalente avec des vérifications de disponibilité des serveurs backend.

4.3 Principe de fonctionnement

Le client n'accède plus directement aux serveurs applicatifs. Il envoie ses requêtes au load balancer, qui choisit ensuite une instance backend parmi celles disponibles. Cette approche permet :

- de répartir la charge sur plusieurs processus ;
- de réduire le risque de saturation d'un seul serveur ;
- d'assurer une meilleure continuité de service ;
- de préparer le système à une architecture plus distribuée.

4.4 Scénarios de test réalisés

Plusieurs scénarios ont été considérés :

1. répartition simple des requêtes entre les serveurs ;
2. charge concurrente avec plusieurs clients ;
3. augmentation progressive de la charge ;
4. panne d'un nœud backend ;
5. reprise du nœud et réintégration dans la rotation.

Les fichiers de logs du load balancer et des serveurs backend permettent de vérifier que les requêtes sont effectivement distribuées entre les différentes instances.

4.5 Captures de la partie load balancing

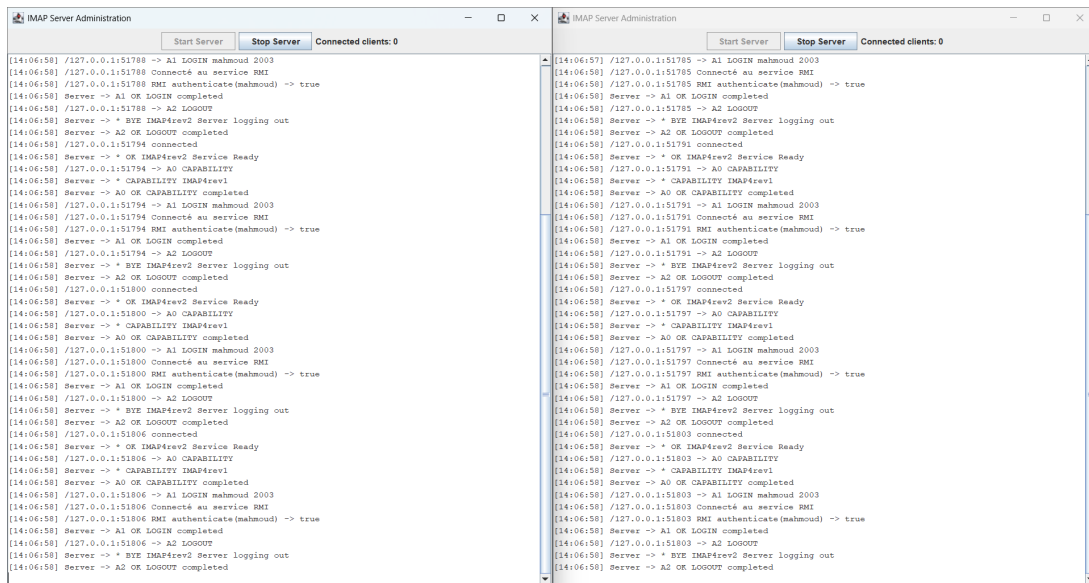


FIGURE 4.1 – Architecture générale de load balancing mise en place.

```
SCENARIO 1 : Repartition Round Robin
Host : localhost:1430 (HAProxy)
10 connexions IMAP successives

=====

[ 1] Connexion ... OK (2874 ms)
[ 2] Connexion ... OK (362 ms)
[ 3] Connexion ... OK (91 ms)
[ 4] Connexion ... OK (74 ms)
[ 5] Connexion ... OK (66 ms)
[ 6] Connexion ... OK (99 ms)
[ 7] Connexion ... OK (135 ms)
[ 8] Connexion ... OK (60 ms)
[ 9] Connexion ... OK (65 ms)
[10] Connexion ... OK (58 ms)

-----

Succes : 10 | Echecs : 0
Temps total : 3954 ms
Temps moyen : 395,4 ms/connexion

Verifiez les fenetres IMAP Instance-1 [1431]
et IMAP Instance-2 [1432] pour voir la distribution.
[10] Connexion ... OK (58 ms)

-----

Succes : 10 | Echecs : 0
Temps total : 3954 ms
Temps moyen : 395,4 ms/connexion

Verifiez les fenetres IMAP Instance-1 [1431]
et IMAP Instance-2 [1432] pour voir la distribution.
et IMAP Instance-2 [1432] pour voir la distribution.

-----

Appuyez sur une touche pour continuer... [ ]
```

FIGURE 4.2 – Configuration NGINX ou lancement du frontal.

```

127.0.0.1 [20/Apr/2026:14:06:57 +0100] TCP 200 -> serveur_choisi=127.0.0.1:1431 (temps=2.737 s)
127.0.0.1 [20/Apr/2026:14:06:57 +0100] TCP 200 -> serveur_choisi=127.0.0.1:1432 (temps=0.361 s)
127.0.0.1 [20/Apr/2026:14:06:58 +0100] TCP 200 -> serveur_choisi=127.0.0.1:1431 (temps=0.089 s)
127.0.0.1 [20/Apr/2026:14:06:58 +0100] TCP 200 -> serveur_choisi=127.0.0.1:1432 (temps=0.074 s)
127.0.0.1 [20/Apr/2026:14:06:58 +0100] TCP 200 -> serveur_choisi=127.0.0.1:1431 (temps=0.065 s)
127.0.0.1 [20/Apr/2026:14:06:58 +0100] TCP 200 -> serveur_choisi=127.0.0.1:1432 (temps=0.099 s)
127.0.0.1 [20/Apr/2026:14:06:58 +0100] TCP 200 -> serveur_choisi=127.0.0.1:1431 (temps=0.134 s)
127.0.0.1 [20/Apr/2026:14:06:58 +0100] TCP 200 -> serveur_choisi=127.0.0.1:1432 (temps=0.058 s)
127.0.0.1 [20/Apr/2026:14:06:58 +0100] TCP 200 -> serveur_choisi=127.0.0.1:1431 (temps=0.065 s)
127.0.0.1 [20/Apr/2026:14:06:58 +0100] TCP 200 -> serveur_choisi=127.0.0.1:1432 (temps=0.057 s)

```

FIGURE 4.3 – Configuration HAProxy ou vue des backends.

```

=====
SCENARIO 2 : Charge concurrente
Lance 5 threads client simultanement
=====

=====
SCENARIO 2 : Charge concurrente
5 clients IMAP simultanes
=====

Client-1 : OK (406 ms)
Client-5 : OK (465 ms)
Client-2 : OK (586 ms)
Client-4 : OK (709 ms)
Client-3 : OK (774 ms)

-----
Succes : 5 | Echecs : 0
Temps total (//): 801 ms
-----

Appuyez sur une touche pour continuer... █

```

FIGURE 4.4 – Démarrage des instances SMTP, POP3 et IMAP.

```

127.0.0.1 [20/Apr/2026:14:11:56 +0100] TCP 200 -> serveur_choisi=127.0.0.1:1431 (temps=0.233 s)
127.0.0.1 [20/Apr/2026:14:11:56 +0100] TCP 200 -> serveur_choisi=127.0.0.1:1431 (temps=0.303 s)
127.0.0.1 [20/Apr/2026:14:11:56 +0100] TCP 200 -> serveur_choisi=127.0.0.1:1431 (temps=0.425 s)
127.0.0.1 [20/Apr/2026:14:11:56 +0100] TCP 200 -> serveur_choisi=127.0.0.1:1432 (temps=0.548 s)
127.0.0.1 [20/Apr/2026:14:11:56 +0100] TCP 200 -> serveur_choisi=127.0.0.1:1432 (temps=0.613 s)

```

FIGURE 4.5 – Démarrage des instances web backend.

```
=====
SCENARIO 3 : Saturation progressive
Increase de 2 a 20 connexions avec mesure du temps
=====

=====
SCENARIO 3 : Saturation progressive
De 2 a 20 clients simultanés
=====

Clients      Succes      Echeecs      Temps moy.
-----
2            2           0           434,0 ms
5            5           0           138,6 ms
10           10          0           213,5 ms
15           15          0           300,5 ms
20           20          0           412,9 ms

-----
Analyse : une augmentation du temps moyen
indique l'approche de la saturation.
-----

Appuyez sur une touche pour continuer... █
```

FIGURE 4.6 – Répartition simple des requêtes entre les nœuds.

```

127.0.0.1 [20/Apr/2026:14:31:05 +0100] TCP 200 -> serveur_choisi=127.0.0.1:1432 (temps=0.366 s)
127.0.0.1 [20/Apr/2026:14:31:05 +0100] TCP 200 -> serveur_choisi=127.0.0.1:1432 (temps=0.405 s)
127.0.0.1 [20/Apr/2026:14:31:05 +0100] TCP 200 -> serveur_choisi=127.0.0.1:1431 (temps=0.447 s)
127.0.0.1 [20/Apr/2026:14:31:05 +0100] TCP 200 -> serveur_choisi=127.0.0.1:1432 (temps=0.480 s)
127.0.0.1 [20/Apr/2026:14:31:05 +0100] TCP 200 -> serveur_choisi=127.0.0.1:1432 (temps=0.511 s)
127.0.0.1 [20/Apr/2026:14:31:05 +0100] TCP 200 -> serveur_choisi=127.0.0.1:1431 (temps=0.543 s)
127.0.0.1 [20/Apr/2026:14:31:06 +0100] TCP 200 -> serveur_choisi=127.0.0.1:1432 (temps=0.107 s)
127.0.0.1 [20/Apr/2026:14:31:06 +0100] TCP 200 -> serveur_choisi=127.0.0.1:1431 (temps=0.073 s)
127.0.0.1 [20/Apr/2026:14:31:06 +0100] TCP 200 -> serveur_choisi=127.0.0.1:1431 (temps=0.120 s)
127.0.0.1 [20/Apr/2026:14:31:06 +0100] TCP 200 -> serveur_choisi=127.0.0.1:1432 (temps=0.151 s)
127.0.0.1 [20/Apr/2026:14:31:06 +0100] TCP 200 -> serveur_choisi=127.0.0.1:1431 (temps=0.184 s)
127.0.0.1 [20/Apr/2026:14:31:06 +0100] TCP 200 -> serveur_choisi=127.0.0.1:1432 (temps=0.248 s)
127.0.0.1 [20/Apr/2026:14:31:06 +0100] TCP 200 -> serveur_choisi=127.0.0.1:1432 (temps=0.262 s)
127.0.0.1 [20/Apr/2026:14:31:06 +0100] TCP 200 -> serveur_choisi=127.0.0.1:1431 (temps=0.306 s)
127.0.0.1 [20/Apr/2026:14:31:06 +0100] TCP 200 -> serveur_choisi=127.0.0.1:1431 (temps=0.363 s)
127.0.0.1 [20/Apr/2026:14:31:06 +0100] TCP 200 -> serveur_choisi=127.0.0.1:1432 (temps=0.400 s)
127.0.0.1 [20/Apr/2026:14:31:06 +0100] TCP 200 -> serveur_choisi=127.0.0.1:1432 (temps=0.435 s)
127.0.0.1 [20/Apr/2026:14:31:06 +0100] TCP 200 -> serveur_choisi=127.0.0.1:1431 (temps=0.478 s)
127.0.0.1 [20/Apr/2026:14:31:06 +0100] TCP 200 -> serveur_choisi=127.0.0.1:1431 (temps=0.514 s)
127.0.0.1 [20/Apr/2026:14:31:06 +0100] TCP 200 -> serveur_choisi=127.0.0.1:1432 (temps=0.544 s)
127.0.0.1 [20/Apr/2026:14:31:07 +0100] TCP 200 -> serveur_choisi=127.0.0.1:1432 (temps=0.578 s)
127.0.0.1 [20/Apr/2026:14:31:07 +0100] TCP 200 -> serveur_choisi=127.0.0.1:1431 (temps=0.602 s)
127.0.0.1 [20/Apr/2026:14:31:07 +0100] TCP 200 -> serveur_choisi=127.0.0.1:1431 (temps=0.646 s)
127.0.0.1 [20/Apr/2026:14:31:07 +0100] TCP 200 -> serveur_choisi=127.0.0.1:1431 (temps=0.597 s)
127.0.0.1 [20/Apr/2026:14:31:07 +0100] TCP 200 -> serveur_choisi=127.0.0.1:1432 (temps=0.624 s)
127.0.0.1 [20/Apr/2026:14:31:07 +0100] TCP 200 -> serveur_choisi=127.0.0.1:1432 (temps=0.667 s)

```

FIGURE 4.7 – Test de charge concurrente.

```
[!] ACTION REQUISE : Fermez manuellement la fenetre
"IMAP Instance-2 [1432]" avec la croix rouge.

Appuyez sur une touche APRES avoir ferme la fenetre...
Appuyez sur une touche pour continuer...

Envoi de 6 connexions vers le LB (doit aller vers Instance-1)...
=====
SCENARIO 1 : Repartition Round Robin
Host : localhost:1430 (HAProxy)
10 connexions IMAP successives
=====

[ 1] Connexion ... OK (2199 ms)
[ 2] Connexion ... OK (42 ms)
[ 3] Connexion ... OK (53 ms)
[ 4] Connexion ... OK (70 ms)
[ 5] Connexion ... OK (126 ms)
[ 6] Connexion ... OK (69 ms)
[ 7] Connexion ... OK (36 ms)
[ 8] Connexion ... OK (70 ms)
[ 9] Connexion ... OK (36 ms)
[10] Connexion ... OK (46 ms)

-----

Succes : 10 | Echecs : 0
Temps total : 2807 ms
Temps moyen : 280,7 ms/connexion

Verifiez les fenetres IMAP Instance-1 [1431]
et IMAP Instance-2 [1432] pour voir la distribution.
-----

Verifiez que toutes les requetes ont ete traitees par Instance-1.
Appuyez sur une touche pour continuer... █
va: Ready
```

FIGURE 4.8 – Observation des temps de réponse ou des journaux de distribution.

```
127.0.0.1 [20/Apr/2026:14:33:06 +0100] TCP 200 -> serveur_choisi=127.0.0.1:1431 (temps=0.040 s)
127.0.0.1 [20/Apr/2026:14:33:07 +0100] TCP 200 -> serveur_choisi=127.0.0.1:1431 (temps=0.052 s)
127.0.0.1 [20/Apr/2026:14:33:07 +0100] TCP 200 -> serveur_choisi=127.0.0.1:1431 (temps=0.069 s)
127.0.0.1 [20/Apr/2026:14:33:07 +0100] TCP 200 -> serveur_choisi=127.0.0.1:1431 (temps=0.125 s)
127.0.0.1 [20/Apr/2026:14:33:07 +0100] TCP 200 -> serveur_choisi=127.0.0.1:1431 (temps=0.068 s)
127.0.0.1 [20/Apr/2026:14:33:07 +0100] TCP 200 -> serveur_choisi=127.0.0.1:1431 (temps=0.035 s)
127.0.0.1 [20/Apr/2026:14:33:07 +0100] TCP 200 -> serveur_choisi=127.0.0.1:1431 (temps=0.068 s)
127.0.0.1 [20/Apr/2026:14:33:07 +0100] TCP 200 -> serveur_choisi=127.0.0.1:1431 (temps=0.035 s)
127.0.0.1 [20/Apr/2026:14:33:07 +0100] TCP 200 -> serveur_choisi=127.0.0.1:1431 (temps=0.046 s)
```

FIGURE 4.9 – Simulation d'une panne d'un serveur backend.

```
[!] ACTION REQUISE : Relancez IMAP Instance-2 avec la commande :
    java -cp "target\classes;target\dependency\*" org.example.ImapServerGUI 1432

Puis appuyez sur Start Server dans la nouvelle fenetre.
Appuyez sur une touche quand Instance-2 est prete...
Appuyez sur une touche pour continuer...

Envoi de 8 connexions (doit alterner entre les 2 instances)...
=====
SCENARIO 1 : Repartition Round Robin
Host : localhost:1430 (HAProxy)
10 connexions IMAP successives
=====

[ 1] Connexion ... OK (148 ms)
[ 2] Connexion ... OK (59 ms)
[ 3] Connexion ... OK (652 ms)
[ 4] Connexion ... OK (38 ms)
[ 5] Connexion ... OK (65 ms)
[ 6] Connexion ... OK (41 ms)
[ 7] Connexion ... OK (60 ms)
[ 8] Connexion ... OK (85 ms)
[ 9] Connexion ... OK (53 ms)
[10] Connexion ... OK (70 ms)

-----

Succes : 10 | Echecs : 0
Temps total : 1328 ms
Temps moyen : 132,8 ms/connexion

Verifiez les fenetres IMAP Instance-1 [1431]
et IMAP Instance-2 [1432] pour voir la distribution.
-----

Verifiez la distribution dans les 2 fenetres IMAP.
Appuyez sur une touche pour continuer... █
```

FIGURE 4.10 – Continuité de service après panne d'un nœud.

```
127.0.0.1 [20/Apr/2026:14:35:30 +0100] TCP 200 -> serveur_choisi=127.0.0.1:1431 (temps=0.058 s)
127.0.0.1 [20/Apr/2026:14:35:31 +0100] TCP 200 -> serveur_choisi=127.0.0.1:1432 (temps=0.652 s)
127.0.0.1 [20/Apr/2026:14:35:31 +0100] TCP 200 -> serveur_choisi=127.0.0.1:1431 (temps=0.038 s)
127.0.0.1 [20/Apr/2026:14:35:31 +0100] TCP 200 -> serveur_choisi=127.0.0.1:1432 (temps=0.064 s)
127.0.0.1 [20/Apr/2026:14:35:31 +0100] TCP 200 -> serveur_choisi=127.0.0.1:1431 (temps=0.040 s)
127.0.0.1 [20/Apr/2026:14:35:31 +0100] TCP 200 -> serveur_choisi=127.0.0.1:1432 (temps=0.055 s)
127.0.0.1 [20/Apr/2026:14:35:31 +0100] TCP 200 -> serveur_choisi=127.0.0.1:1431 (temps=0.085 s)
127.0.0.1 [20/Apr/2026:14:35:31 +0100] TCP 200 -> serveur_choisi=127.0.0.1:1432 (temps=0.051 s)
127.0.0.1 [20/Apr/2026:14:35:32 +0100] TCP 200 -> serveur_choisi=127.0.0.1:1431 (temps=0.070 s)
```

FIGURE 4.11 – Réintégration d'une instance après redémarrage.

```
=====
TEST NGINX WEB API - Port 80 (HTTP)
100 requetes, 10 concurrentes
NGINX doit distribuer entre instances 8080 et 8081
=====

Lancement du simulateur de charge (equivalent a Apache Bench)...
=====
SCENARIO 6 : Test NGINX Web API (HTTP)
100 requetes HTTP / 10 concurrentes
Cible : http://localhost:80/api/messages
=====

-----
Requetes reussies : 0 / 100
Erreurs web       : 100
Temps d'execution : 20416 ms
Temps moyen/req   : 2039,69 ms
Debit             : 4,90 requetes/sec
-----

Regardez la fenetre ou tourne NGINX !
Vous pouvez ouvrir le fichier E:\nginx-1.30.0\logs\access.log
pour voir la distribution entre les ports 8080 et 8081.

Appuyez sur une touche pour continuer... █
```

FIGURE 4.12 – Validation finale du comportement du système sous load balancing.

4.6 Analyse et discussion

La mise en place du load balancing constitue une évolution importante du projet car elle transforme une architecture initialement centralisée en une architecture plus robuste et plus proche d'un environnement distribué réel.

Les principaux apports sont :

- une meilleure répartition de la charge ;
- une meilleure disponibilité des services ;
- une plus grande facilité pour réaliser des tests de montée en charge ;
- une base commune garantissant la cohérence des données entre les instances.

Les limites observées concernent principalement :

- la gestion avancée des sessions ;

- l'absence d'un déploiement sur plusieurs machines physiques ;
- le besoin d'ajouter des mécanismes de supervision plus poussés ;
- la nécessité de tests de charge plus massifs avec des outils comme JMeter ou Apache Bench.

Conclusion générale

Au terme de ce travail, le système de messagerie distribuée a été enrichi de manière significative. L'intégration d'une base MySQL a permis de centraliser la persistance des données. L'ajout d'une interface web a rendu le système plus accessible. Le développement de clients avec JavaMail a validé l'interopérabilité avec les protocoles supportés. Enfin, la mise en place du load balancing a introduit une dimension de scalabilité et de tolérance aux pannes.

Ce projet illustre ainsi plusieurs notions fondamentales des systèmes distribués : la séparation des services, la communication inter-processus, la centralisation de la persistance, l'exposition d'API, ainsi que la répartition de charge. Il constitue une base solide pour des améliorations futures telles que l'ajout du chiffrement TLS, la gestion des pièces jointes, l'orchestration par conteneurs ou encore l'automatisation du déploiement.

A

Remarques pour Overleaf

- Ajouter le fichier `rapport_tp_messagerie.tex` dans le projet Overleaf.
- Ajouter aussi le dossier contenant les captures nommé `rapport tp`.
- Si Overleaf pose un problème avec l'espace dans le nom du dossier, renommer simplement le dossier en `rapport_tp` puis remplacer dans ce fichier toutes les occurrences de `rapport tp/` par `rapport_tp/`.
- Les légendes peuvent être modifiées directement selon le contenu exact de chaque capture.